

```

flowchart LR
%% Parameters (from config)
subgraph Config
gamma[reputation_decay  $\gamma = 0.75$ ]
hsize[history_size = 150]
th[anomaly_threshold  $\tau = 0.6$ ]
end

%% Per-client state (bounded)
subgraph State[Per-Client State]
R[[Reputation  $R \in [0,1]$ ]]
H[[History H: circular buffer  $\leq 150$  events]]
end

E[Incoming Event\n(client_id, features, outcome, ts)]
E --> L[Load (R, H) for client_id]

L --> D[Decay reputation\n $R := \gamma \cdot R$ ]
D --> F[Extract features from event and H\n(e.g., deviations, burstiness, error rate)]
F --> S[Compute anomaly score  $s \in [0,1]$ ]

S -->| $s > \tau$ | ANOMALY[Anomalous event]
S -->| $s \leq \tau$ | NORMAL[Normal event]

%% Impact computation (policy/model-defined)
ANOMALY --> I1[Compute impact  $\Delta_t \leq 0$ \n(heavier penalty for higher s)]
NORMAL --> I2[Compute impact  $\Delta_t \geq 0$  or small negative\n(reward or small drift)]

I1 --> U
I2 --> U

U[Update reputation\n $R := \text{clip}(R + \Delta_t, 0, 1)$ ]
U --> W[Write-back R]
W --> HP[Append event to H; if  $|H| > 150$  drop oldest]

%% Decision policy uses both s and R
U --> P[Policy decision]
S --> P

P -->|High s or Low R| ACT1[Block / Throttle / Challenge]
P -->|Otherwise| ACT2[Allow / Fast-path]

%% Notes
classDef note fill:#f9f9f9,stroke:#bbb,color:#333,font-size:12px;
N1{{Per-event decay avoids idle-time inflation of R}}:::note
N2{{clip(x,0,1) enforces bounds on reputation}}:::note
N3{{History H bounds memory and enables sliding-window stats}}:::note

```

D -.-> N1
U -.-> N2
HP -.-> N3